CS3230 Design and Analysis of Algorithms	August 21, 2019

Solution *Sketches* to Problem Set 1

Name: Divesh Aggarwal

Problem 1-1 (Tower of Hanoi)

The Tower of Hanoi is a well known mathematical puzzle. It consists of three rods, and a number n of disks of different sizes which can slide onto any rod. The puzzle starts with all disks stacked up on the 1st rod in order of increasing size with the smallest on top. The objective of the puzzle is to move all the disks to the 3rd rod, while obeying the following rules.

- Only one disk is moved at a time
- Each move consists of taking one disk from top of a rod, and moving it on top of the stack on another rod
- No disk may be placed on top of a smaller disk.

A recursive algorithm that solves this problem is as follows: We first move the top $n-1$ disks from rod 1 to rod 2. Then we move the largest disk from rod 1 to rod 3 and then move the $n-1$ smaller disks from rod 2 to rod 3. Using the symmetry between the rods, the number of steps that this algorithm takes is given by the recurrence

$$T(n) = 2T(n-1) + 1 \ ,$$

which can be solved to get $T(n) = 2^n - 1$.

(a) Show that the above algorithm is optimal, i.e., there does not exist a strategy that solves the Tower of Hanoi puzzle in less than $2^n - 1$ steps.

Solution: Let $S(n)$ be the smallest number of steps needed to solve the puzzle for n disks. Let A be the algorithm that achieves this smallest number of steps. Let the k-th step be the first time in A when the largest disk is moved from rod 1. At this step, the remaining $n-1$ disks should all be stacked on one rod, since the largest disk cannot be moved on top of another disk. Thus,

$$k - 1 \geq S(n-1) \ .$$

Similarly, after the k-th step, there should be at least $k-1$ steps for stacking up the $k-1$ smaller disks on top of the largest disk. Thus,

$$S(n) \geq k + S(n-1) \ .$$

This implies that $S(n) \geq 2S(n-1) + 1$, which solves to $S(n) \geq 2^n - 1$. □

(b) Suppose the moves are restricted further such that you are only allowed to move disks to and from rod 2. Give an algorithm that solves the puzzle in $O(3^n)$ steps.

Solution: We first move the top $n-1$ disks from rod 1 to rod 3. Then we move the largest disk from rod 1 to rod 2 and then move the $n-1$ smaller disks from rod 3 to rod 1. Then move the largest disk from rod 2 to rod 3, and finally move the $n-1$ smallest disks from rod 1 to rod 3. Using the symmetry between rod 1 and rod 3, the number of steps that this algorithm takes is given by the recurrence

$$\begin{aligned} T(1) &= 2 \\ T(n) &= 3T(n-1)+2\ , \end{aligned}$$

which can be solved to get $T(n) = 3^n - 1$. □

(c) Suppose the moves are restricted such that you are only allowed to move from rod 1 to rod 2, rod 2 to rod 3, and from rod 3 to rod 1. Give an algorithm that solves the puzzle in $O((1+\sqrt{3})^n)$ steps.

Solution: Let $T(n)$ be the number of steps taken for moving n disks from rod 1 to rod 3. Note that by symmetry, this is also the number of steps taken for moving n disks from rod 3 to rod 2 and from rod 2 to rod 1. Also, let $S(n)$ be the number of steps taken for moving n disks from rod 1 to rod 2 (and by symmetry, from rod 2 to rod 3 and from rod 3 to rod 1).

Consider the following recursive algorithm for moving n disks from rod 1 to rod 3. Move $n-1$ disks from rod 1 to rod 3, then move the largest disk to rod 2. Then move $n-1$ disks from rod 3 to rod 1 followed by moving the largest disk to rod 3. Finally, move the $n-1$ disks from rod 1 to rod 3. This gives us the recurrence

$$T(n) = 2T(n-1) + S(n-1) + 2\ . \tag{1}$$

Consider the following recursive algorithm for moving n disks from rod 1 to rod 2. Move $n-1$ disks from rod 1 to rod 3, then move the largest disk to rod 2. Then move $n-1$ disks from rod 3 to rod 2. This gives us the recurrence

$$S(n) = 2T(n-1) + 1\ . \tag{2}$$

Combining equation (1) and (2), we get

$$T(n) = 2T(n-1) + 2T(n-2) + 3\ . \tag{3}$$

Also, $T(1) = 2$. For solving this, let $R(n) = T(n) + 1$. Then, $R(n) = 2R(n-1) + 2R(n-2)$. We can show by induction, that $R(n) \leq 2(1+\sqrt{3})^n$.

Alternative solution: Let us try to obtain a direct combinatorial argument to arrive at the recurrence in eq (3). We let the disks be numbered according to their size, i.e., disk 1 is the smallest, and disk n the largest. Suppose we follow the following steps:

- Move disks $1, \ldots, n-1$ disks from rod 1 to rod 3 in $T(n-1)$ steps.
- Move disk n from rod 1 to rod 2
- Move disks $1, \ldots, n-2$ from rod 3 to rod 2 in $T(n-2)$ steps (Rod 2 now has disks $1, 2, \ldots, n-2, n$).

- Move disk $n-1$ from rod 3 to rod 1.
- Move disks $1, \ldots, n-2$ from rod 2 to rod 1 in $T(n-2)$ steps
- Move disk n from rod 2 to rod 3.
- Move disks $1, \ldots, n-1$ from rod 1 to rod 3 in $T(n-1)$ steps.

This requires a total of $2T(n-1) + T(n-2) + 3$ steps. □

Problem 1-2 (Counting Inversions)

Let $A[1 \ldots n]$ be an array of pairwise different numbers. We call pair of indices $1 \leq i < j \leq n$ an *inversion* of A if $A[i] > A[j]$. The goal of this problem is to develop a divide-and-conquer based algorithm running in time $\Theta(n \log n)$ for computing the number of inversions in A.

(a) Suppose you are given a pair of *sorted* integer arrays A and B of length $n/2$ each. Let C an n-element array consisting of the concatenation of A followed by B. Give an algorithm (in pseudocode) for counting the number of inversions in C and analyze its runtime. Make sure you also argue (in English) why your algorithm is correct.

Solution:

```
MergeCount(A[1 ... n/2], B[1 ... n/2])
    i = j = 1
    count = 0
    While (i ≤ n/2) Do
        While (j ≤ n/2) ∧ (A[i] > B[j]) Do
            j = j + 1
        count = count + j − 1
    Return count
```

The code is correct because i iterates over all elements of A. For each such i, the inner while loop points j at the smallest element of B that is greater than or equal to $A[i]$. That is once this loop completes we know that $B[1]$ through $B[j-1]$ are all smaller then $A[i]$. In other words there are $j-1$ inversions for that i. Thus at the end of the outer while loop *count* is exactly the number of inversions of the type where the first element is in A and the second in B. Now since both arrays are sorted we know there can only be inversions of that type in the concatenation of A and B and so the algorithm is correct.

Although we have a double loop the runtime is bounded by the fact that j is only ever incremented and the inner loop does nothing when $j > n/2$. Thus the algorithm runs in time $O(n)$. □

(b) Give an algorithm (in pseudocode) for counting the number of inversions in an n element array A that runs in time $\Theta(n \log n)$. Make sure you formally prove that your algorithm runs in time $\Theta(n \log n)$ (e.g., write the recurrence and solve it.)
(**Hint**: Combine Merge Sort with part (a).)

Solution: The algorithm makes direct use of the the MERGE function from merge sort and the MERGECOUNT from part (a).

COUNTINVS$(C[1 \dots n])$
  If $n = 1$ **Return** $(0, C)$
  $(l, A[1 \dots \frac{n}{2}]) =$ COUNTINVS$(C[1 \dots \frac{n}{2}])$
  $(r, B[1 \dots \frac{n}{2}]) =$ COUNTINVS$(C[\frac{n}{2} + 1 \dots n])$
  $count = l + r +$ MERGECOUNT(A, B)
  Return $(count,$ MERGE$(A, B))$

Note: the functions MERGECOUNT and MERGE could of course be combined into a single procedure. But since they both take time $O(n)$ the complexity of such a procedure is not significantly different then simply running the two functions consecutively.

One might ask why COUNTINVS returns a (sorted) array along with the number of inversions. For example, as an alternative it might be sufficient to only return the number of inversions and then call MERGESORT on A and B before combining the solutions with MERGECOUNT. However this would now mean that the "conquer" step takes time $O(n \log n)$ rather then just $O(n)$. In particular we would be sorting the same sequence multiple times. For example $A[1 \dots n/4]$ would be sorted twice this way.

The recurrence equation for this algorithm is:

$$T(n) = 2T(n/2) + O(n).$$

We solve this with the master theorem. $a = 2$, $b = 2$ and $f(n) = n$. Thus we are in the second case of the master theorem since $f(n) = n = \Theta(n) = \Theta(n^{\log_b a})$. Therefore we get that $T(n) = \Theta(n \log n)$. □

Problem 1-3 (Counting Intersections)

Suppose you are given two sets of n points, one set $p_1, p_2, ..., p_n$ on the line $y = 0$ and the other set $q_1, q_2, ..., q_n$ on the line $y = 1$. Create a set of n line segments by connecting each point p_i to the corresponding point q_i. Describe and analyze a divide-and-conquer algorithm to determine how many pairs of these line segments intersect, in $O(n \log n)$time. [Hint:See the previous problem.]

Solution: Let us first formally define when an intersection should occur. Let's say that (p_1, q_1) and (p_2, q_2) are two lines, and then notice that there is an intersection between these two lines if and only if (without loss of generality) $p_1 < p_2$ whilst $q_1 > q_2$. In fact, this is very similar to the problem of counting inversions, since what one can do is think of the p points as the indices, and the corresponding q points as the values. An example of this is illustrated in figure 1.

Thus what we can do is transform our input of two lines into an array of integers, where we order the points q_i based on their corresponding order of points p_i.

Then the analysis and correctness and runtime follow suit as is in the previous question. □

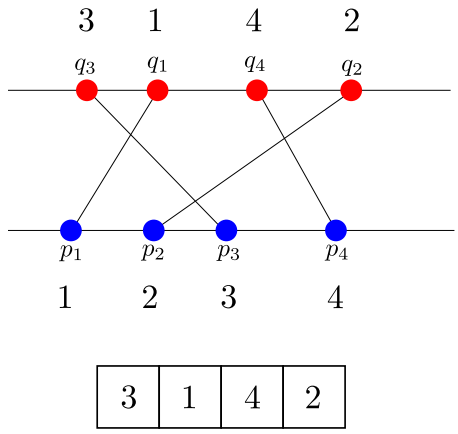

Figure 1: Example

Problem 1-4 (Fibonacci Numbers)

The Fibonacci number $\{F_n \mid n \geq 0\}$ are defined as follows: $F_0 = 0$, $F_1 = 1$ and, for $i > 1$, define $F_i := F_{i-1} + F_{i-2}$.

(a) Notice the following matrix equation:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} F_i \\ F_{i-1} \end{pmatrix} = \begin{pmatrix} F_{i+1} \\ F_i \end{pmatrix}$$

Use this equation and the ideas of fast $O(\log n)$ time exponentiation to build an efficient algorithm for computing F_n. Analyze it's runtime in terms of the number of 2×2 matrix multiplications performed (each of which takes a single integer addition).

Solution: Let I be the 2×2 identity matrix, $F := \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$ and let $A := \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$. We use the insight that $\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = A^n \cdot F$. So to compute F_n the algorithm first computes A^n, multiplies that with F and then outputs the first component of the result. In pseudocode:

```
FIB(n)
    A_n = AEXP(n)
    H = A_n · F
    Return H[1]

AEXP(n)
    If (n = 0)
        Return I
    If (n mod 2 = 1)        // "if n is odd"
        Return A*AEXP(n − 1)
    Else
        Return (AEXP(n/2))^2
```

We measure runtime in the number of matrix multiplications. Let $T(n)$ be the runtime of AExp(n), then $T(n) = T(n) + \theta(1)$ so $T(n) = O(\log n)$. $\square$

(b) Prove by induction that, for some constant $a > 1$, $F_n \leq a^n$. Thus, F_n takes $O(n)$ bits to represent. What is the smallest constant a you can find?

Solution: Base case is trivial. Let $F_{n-2} \leq a^{n-2}$ and $F_{n-1} \leq a^{n-1}$
Now, $F_n = F_{n-1} + F_{n-2} \leq a^{n-1} + a^{n-2} = a^{n-2}$ (1+a) $\leq a^{n-2} * a^2 = a^n$
when $a + 1 \leq a^2$ or, $a \geq \frac{1+\sqrt{5}}{2} = \phi \approx 1.618$. $\square$

(c) In your algorithm of part (a) you only counted the number of 2×2 matrix multiplications. However, the integers used to compute (F_{i+1}, F_i) from (F_i, F_{i-1}) are $O(i)$ bits long, by part (b). Thus, the matrix-vector multiplication takes "real time" $O(i)$. Analyze the actual running time $T(n)$ of your algorithm in part (a). Use Karatsuba's algorithm to obtain an optimal answer.

Solution: Now consider the algorithm from part (a). In particular look at the top left component of A^i. For each successive value of i the component requires at least 1 more bit to write down. So for A^n at least n bits are needed and so the algorithm must run in at least $\Omega(n)$ time simply to read this number. But the actual running time is more, as we argue next.

The discrepancy between the runtimes comes from the fact that we didn't take the size of the values into consideration and instead assumed that any and all multiplications take time $O(1)$. But for large terms this is an unrealistic assumption, as multiplication of two $n/2$-bit integers takes time $O(n^{\log_2 3})$ if we use Karatsuba's multiplication.

More specifically, the entries of each of the matrices in the conquer step has $\frac{n}{2}$ bits. So during matrix multiplication in the conquer step, it takes $O(n^2)$ steps to multiply the 2×2 matricies.[1]

Thus, $T(n) = T(n/2) + O(n^{\log_2 3})$, which solves to $T(n) = O(n^{\log_2 3})$. $\square$

Problem 1-5 (Missing String)

(a) Let $n = 2^l - 1$ for some positive integer l. Suppose someone claims to hold an unsorted array $A[1..n]$ of *distinct* l-bit strings; thus, exactly one l-bit string does *not* appear in A. Suppose further that the **only** way we can access A is by calling the function FETCHBIT(i,j), which returns the jth bit of the string $A[i]$ in $O(1)$ time. Describe an algorithm to find the missing string in A using only $O(n)$ calls to FETCHBIT. (**Hint**: It might be easier to recurse on l.)

Solution: First of all, we show how to partition the problem into smaller problems. Without loss of generality, assume that the first bit of the missing string is ZERO. If we look at the first bit of all n strings, there are exactly $(2^{l-1} - 1)$ ZEROs and 2^{l-1} ONEs. Let A' be the array containing all strings whose first bit is ZERO. Moreover, we remove the first bit from

[1] If we only computed A^{i+1} from A^i, this would take time $O(i)$, as we only add i-bit numbers. But to compute $A^{2i} = A^i \times A^i$, we really need to multiply i-bit integers.

all strings in A'. Then A' has exactly $(2^{l-1}-1)$ distinct strings of length $(l-1)$. The missing string in A', appended with an ZERO, is the missing string in A.
Hence, we have the following recursive algorithm:

(1) Query the first bit for all strings in A. That is, $\forall i \in \{1, \ldots, n\}$, call FETCHBIT(i,1).
(2) Let $b \in \{0, 1\}$ be the bit that occurs $(2^{l-1}-1)$ times i.e. b is the first bit of the missing string. Let A' be the array containing all strings whose first bit is b and remove the first bit from all strings in A'. Find the missing string s in A' recursively.
(3) Append b to s and output the string.

Notice that step (1) makes $n = 2^l - 1$ calls to FETCHBIT. Hence, we could write the recurrence as:

$$T(1) = 1$$
$$T(l) \leq T(l-1) + 2^l$$

or equivalently, if we use n for recurrence:

$$T(1) = 1$$
$$T(n) \leq T\Big(\frac{n+1}{2} - 1\Big) + n$$

Solving the recurrence gives us $T(n) = O(n)$. □

(b) Now suppose $n = 2^l - k$ for some positive integer l and k, and again we are given an array $A[1..n]$ of *distinct* l-bit strings. Describe an algorithm to find the k strings that are missing from A using only $O(n \log k)$ calls to FETCHBIT.

Solution: Similar to (a), we partition the problem into smaller problems by inspecting the first bit of all strings:

(1) Query the first bit of all strings in A. Suppose that there are $(2^{l-1} - p)$ ZEROs and $(2^{l-1} - (k-p))$ ONEs.
(2) Let A_0 be the array containing all strings that starts with ZERO, and remove the first bit from all strings. A_0 has $2^{l-1} - p$ distinct strings of length $(l-1)$. Find the p missing strings from A_0 recursively. Output the strings appended with ZERO.
(3) Similarly define A_1 on strings that starts with ONE and find the missing strings from A_1 recursively.

Now we have the following recurrence defined on l and k.

$$T(l, 0) = 0$$
$$T(l, 1) = O(2^l)$$
$$T(l, k) \leq \max_p \Big(T(l-1, p) + T(l-1, k-p)\Big) + 2^l$$

While the recurrence might be difficult to solve, we can verify that $T(l,k) = O(2^l \log(k+1)) = O(n \log k)$ is the correct answer to the above recurrence, which is sufficient for the purpose of this question. More specifically, for the last recurrence, we have:

$$\begin{aligned}
T(l,k) &\leq \max_p \Big(T(l-1,p) + T(l-1,k-p)\Big) + 2^l \\
&\leq \max_p \Big(c2^{l-1}\log p + c2^{l-1}\log(k-p)\Big) + 2^l \\
&\leq \max_p \Big(c2^{l-1}\log\big(p(k-p)\big)\Big) + 2^l \\
&\leq c2^{l-1}\log(\frac{k}{2})^2 + 2^l \\
&\leq c2^l(\log k - 1) + 2^l \\
&\leq c2^l \log k = O(n \log k)
\end{aligned}$$

$\square$